NATIONAL INSTITUTE OF TECHNOLOGY
AGARTALA,JIRANIA, AGARTALA , TRIPURA (W) ,
799046

8-BIT Binary Calculator Using Logic Gates And 4-BIT Adders With 7 Segment Display Output

DEPARTMENT : ELECTRONICS & INSTRUMENTATION ENGINEERING

Name	Enrollment No.
Tanishq Raj V Rathore	23UEI020
Mrinmoy Biswas	23UEI066
Aditya Kar	23UEI071
Raj Anand	23UEI082
Indrani Chakarborty	23UEI088

Swaprup Mukharjee	23UEI207
Ankit Kumar	23UEI213

Sl.No	Topic
1.	Contents
2.	Abstract
3.	Introduction
4.	Literature Survey
5.	Methodology
6.	Components
7.	Implementation
8.	Result & Discussion
9.	Conclusion
10.	References
11.	Appendix

ABSTRACT

This project is all about building a simple Binary Calculator using basic electronic components like logic gates and 4-bit adders. The goal is to show how computers actually do math at the hardware level — without using any software or microcontrollers. Instead, we use switches to input binary numbers and LEDs or displays to show the results.

At the heart of the calculator are 4-bit binary adders, made from logic gates like AND, OR, XOR, and NOT. These gates are connected in a way that they can add two binary numbers together, just like how it's done inside real digital circuits. For subtraction, we use a trick called two’s complement, which lets us do subtraction using the same adder circuit.This project gives a hands-on experience of how binary numbers and digital logic work together to perform real calculations. It's a great way to understand the basics of how computers and digital devices handle numbers behind the scenes.

INTRODUCTION

What It’s About

This project is all about building a simple and functional **Binary Calculator** using **logic gates** and **4-bit adders**. It’s designed to show how basic arithmetic operations like **addition** and **subtraction** are done inside digital systems — without using any programming or microcontrollers. Instead, we rely entirely on hardware components to carry out binary math, just like early digital computers and parts of modern processors still do.

How It Works

The core of the calculator is built using **logic gates** like AND, OR, XOR, and NOT. These gates are combined to form **1-bit full adders**, which are then connected to create a **4-bit adder circuit**. Two binary numbers are entered using **toggle switches**, and the output — the sum or difference — is shown on **LEDs or a seven-segment display**. For subtraction, we use the **two’s complement method**, which cleverly turns subtraction into addition by flipping the bits and adding one.

Why It Matters

This project gives us a **hands-on understanding of how digital electronics really work**. Instead of just seeing numbers appear on a screen, we get to build the circuit that makes it happen. It’s a fun and practical way to explore how computers perform calculations at the most basic level. Projects like this build a strong foundation for more advanced

topics, including **ALUs, CPUs, and embedded systems**, and help develop both logical thinking and circuit design skills.

Literature Survey

Reversible Logic-Based Adders

In the world of **quantum and low-power computing**, reversible logic is a growing field. The paper *"Design of Efficient Reversible Logic-Based Binary and BCD Adder Circuits"* (arXiv) explores how binary and BCD adders can be built to reduce **energy loss**, **quantum cost**, and **delay**. These designs are key for future tech like **quantum processors** and **ultra-efficient chips**.

Minimal Depth Binary Adders

For systems that prioritize speed, reducing the number of steps (or **depth**) in binary addition is crucial. The research paper *"Binary Adder Circuits of Asymptotically Minimum Depth, Linear Size, and Fan-Out Two"* presents new adder designs that are both **fast and compact**, making them ideal for **VLSI and high-performance computing**.

These studies show how simple concepts like binary addition are being pushed to new limits in cutting-edge technology.

METHODOLOGY

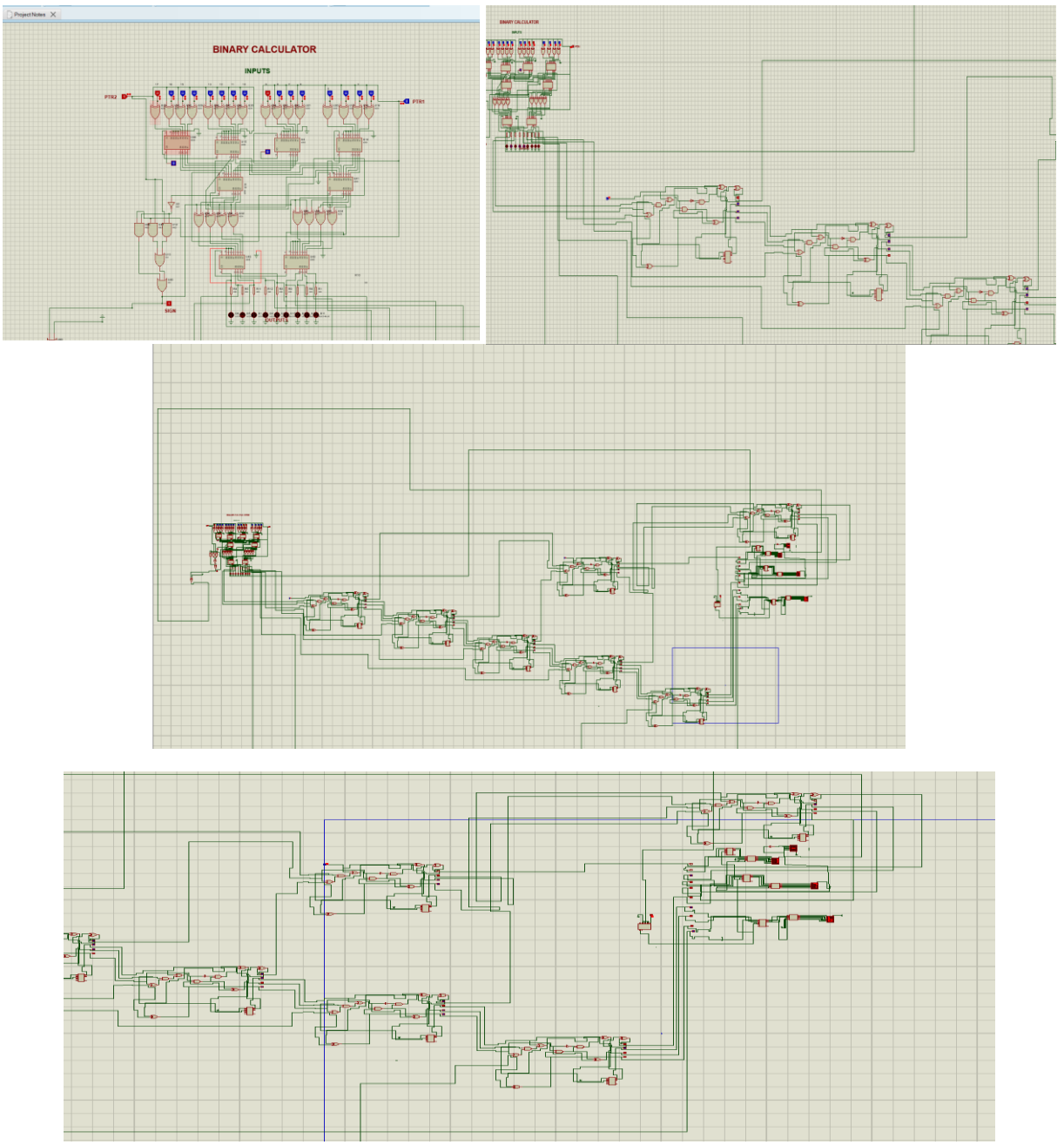

Fig :- Circuit Diagram of the System

METHODOLOGY

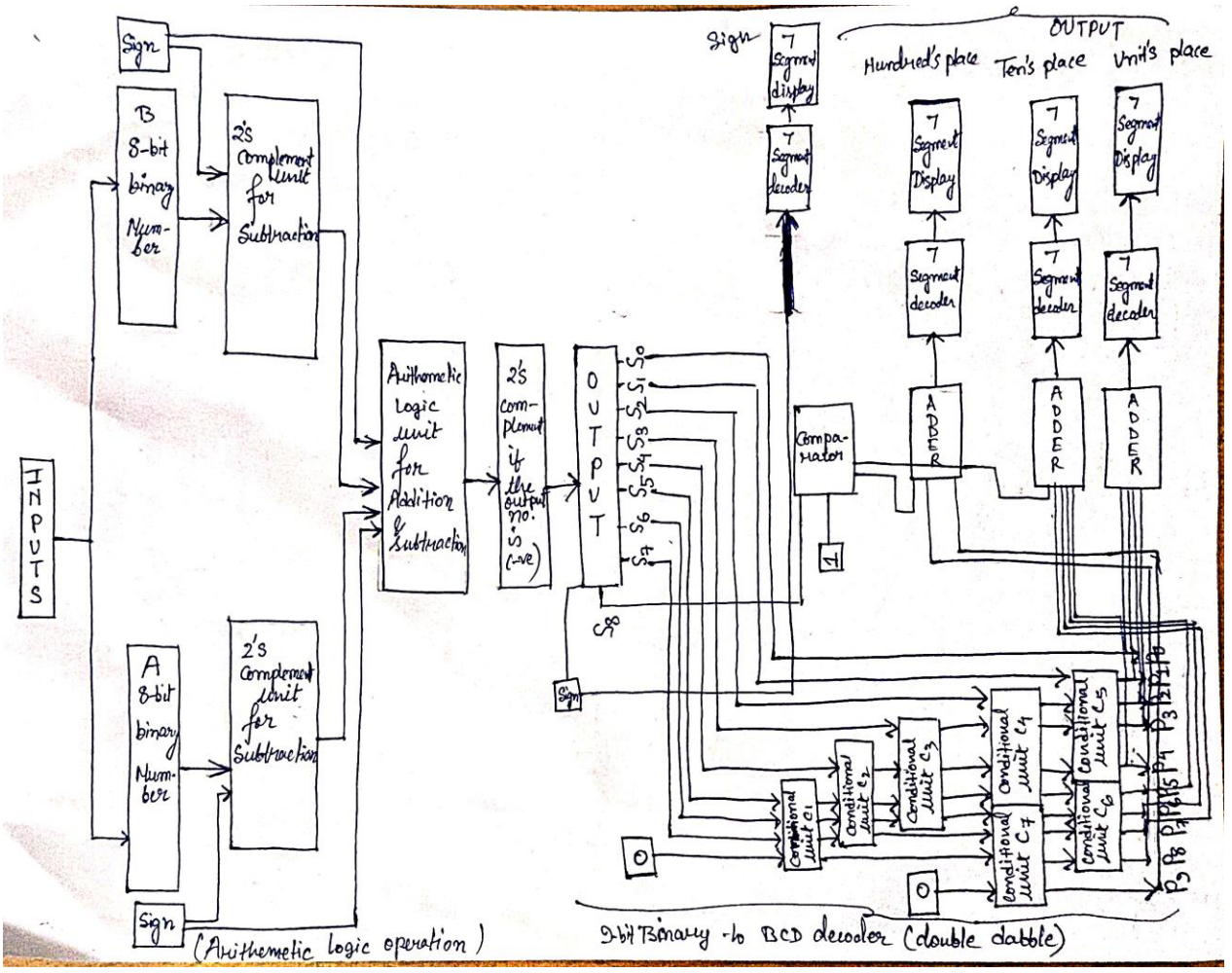

Figure : Block Diagram of the System

COMPONENTS

Sl. No.	Name of Components	Specification	Quantity
1.	IC 7408	AND Gate	
2.	IC 7432	OR Gate	
3.	IC 7486	XOR Gate	
4.	IC 7404	NOT Gate	
5.	IC 74LS83	4-bit full Adder	
6.	IC 74LS47	BCD to 7-Segment Decoder	
7.	7 Segment Display	**----**	
8.	2 Way Dip Switch	Input Devices	
9.	Resistors	1K ohm	
10.	LED		
11.	Breadboard & Wires		As Required

IMPLEMENTATION

To build our 8-bit binary calculator, we used logic gate ICs and other basic components—no microcontrollers or programming involved. The calculator takes two 8-bit binary numbers as input, adds them together, and shows the result on two 7-segment displays in decimal form. Here's how we put it all together:

1. Input Section: We used DIP switches to give binary input. Each switch represents one bit, so we needed 8 switches for the first number and 8 more for the second. These switches let us easily change the values we want to add.

2. Addition Logic: We used two 4-bit adder ICs (like 74LS83) to perform the addition. The first IC adds the lower 4 bits, and the second adds the upper 4 bits. The carry from the first one is connected to the second one, so they work together like an 8-bit adder. The result is an 8-bit binary sum.

3. Converting to Decimal: Since 7-segment displays can’t directly show binary, we converted the binary output into decimal using a binary to BCD (Binary-Coded Decimal) circuit. This part was built using logic gates and follows the Double Dabble algorithm, which breaks the binary number into tens and ones digits in BCD format.

4. Displaying the Result: Once we had the BCD output, we sent it to two decoder ICs (like 74LS47), which translate BCD into signals that control 7-segment displays. One display shows the tens digit and the other shows the ones digit of the result.

5. Overflow Indicator: To handle cases where the result is more than 255 (the max for 8 bits), we added a small LED. If there's a carry-

out from the highest bit, the LED turns on—letting us know the result has overflowed beyond 8 bits.

6. Final Setup: We assembled everything on a breadboard and used jumper wires to connect the components. The entire circuit runs on a 5V power supply. We tested it with different binary inputs and confirmed that the displays correctly showed the decimal result every time.

This calculator is a great example of how we can do real calculations just using logic gates and basic ICs—no need for a microcontroller or computer code.

RESULT & DISCUSSION

After putting the whole circuit together, we tested the binary calculator — and it worked just like we hoped! When we entered two 4-bit binary numbers using the switches, the result showed up correctly on the LEDs. It was really satisfying to see the numbers we calculated on paper match what the circuit gave us.

Addition

The calculator could easily handle binary addition. For example, when we gave `0100` (which is 4) and `0011` (which is 3), the output showed `0111` (7), just as expected. When the result went over 4 bits, the **carry-out LED** turned on — which was a nice way to see overflow in action.

Subtraction

We used the **two's complement method** to handle subtraction. Even though it sounds complicated at first, it worked really well. The calculator gave the correct answers, and we could even see how negative numbers are stored in binary format. That was pretty cool to watch happen live on the LEDs.

Our Takeaway

Seeing logic gates do actual math was eye-opening. It's one thing to learn the theory, but building it and seeing it work made everything click. The project also made us realize how these simple components come together to form the brain of a computer.

Even though it was a small 4-bit calculator, it gave us a huge boost in understanding how real-world processors perform calculations behind the scenes.

CONCLUSION

This project was a great way to really get our hands into the world of digital electronics. By building a simple binary calculator using just **logic**

gates and **4-bit adders**, we got to see how basic math actually happens inside a computer — not through code, but through pure hardware.

It was especially cool to see how subtraction works using the **two's complement method** — a neat trick that shows how clever circuit design can be. Watching the numbers light up on LEDs made everything feel real and gave us a deeper understanding of what's usually hidden inside black boxes.

Overall, this project helped us connect theory with practice. It showed us how even the most advanced systems, like CPUs, start from something simple. And now, we feel more confident about exploring more complex topics like ALUs and microprocessors in the future.

Future Enhancements

- Add subtraction, multiplication, and division operations using additional logic circuits.
- Support signed numbers by implementing two's complement logic.
- Upgrade display with more digits or use an LCD for better readability.
- Include memory/registers to store inputs or results.
- Add error detection and overflow indicators using LEDs or messages.
- Implement operation mode selection (e.g., Add/Subtract toggle switch).
- Migrate the design to FPGA for speed, compact size, and advanced features.

REFERENCES

1. **M. Morris Mano, Michael D. Ciletti**
   Digital Design (5th Edition), Pearson Education, 2013.A fundamental textbook on digital circuits, logic gates, and binary arithmetic.
2. **R.P. Jain**
   Modern Digital Electronics (4th Edition), McGraw-Hill Education, 2009.Covers foundational concepts in combinational and sequential logic including adder design.
3. **Arun Kumar Singh, Yogesh Kumar Sharma**
   "Design of Efficient Reversible Logic Based Binary and BCD Adder Circuits", arXiv:1609.06142 [cs.ET], 2016.

https://arxiv.org/abs/1609.06142
Discusses low-power reversible logic adders for quantum computing.

4. **Krapchenko, A. K.**
   "Binary Adder Circuits of Asymptotically Minimum Depth, Linear Size, and Fan-Out Two", 1967.Foundational work on optimizing binary adder depth and structure.
5. **Thomas L. Floyd**
   Digital Fundamentals (11th Edition), Pearson, 2015.Beginner-friendly reference for learning how binary systems and logic gates function.

APPENDIX